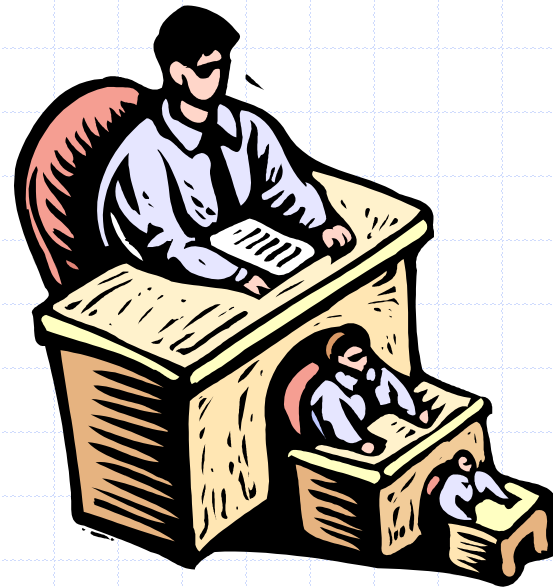


Recursion



The Recursion Pattern

- Classic example – the factorial function:

$$n! = 1 \cdot 2 \cdot 3 \cdot \cdots \cdot (n-1) \cdot n$$

- Recursive definition:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n-1) & \text{else} \end{cases}$$

```
1. int factorial(int n)
2. {
3.     if (n == 0)           // base case
4.         return 1;
5.     else if (n > 0)       // recursive case
6.         return n * factorial(n-1);
7. }
```

Content of a Recursive Method

□ Base case(s)

- Values of the input variables for which we perform no recursive calls are called **base cases** (there should be at least one base case).
- Every possible chain of recursive calls **must** eventually reach a base case.

□ Recursive calls

- Calls to the current method.
- Each recursive call should be defined so that it makes progress towards a base case.

A Perspective on Recursion

1. Decomposition

- Decompose the problem into smaller identical problems

2. Base case

- Smallest problem with known solution

3. Composition

- Compose the solutions of smaller problems

The Recursion Pattern

- ❑ Decomposition into smaller problems
- ❑ Base case: smallest problem
- ❑ Composition of solutions

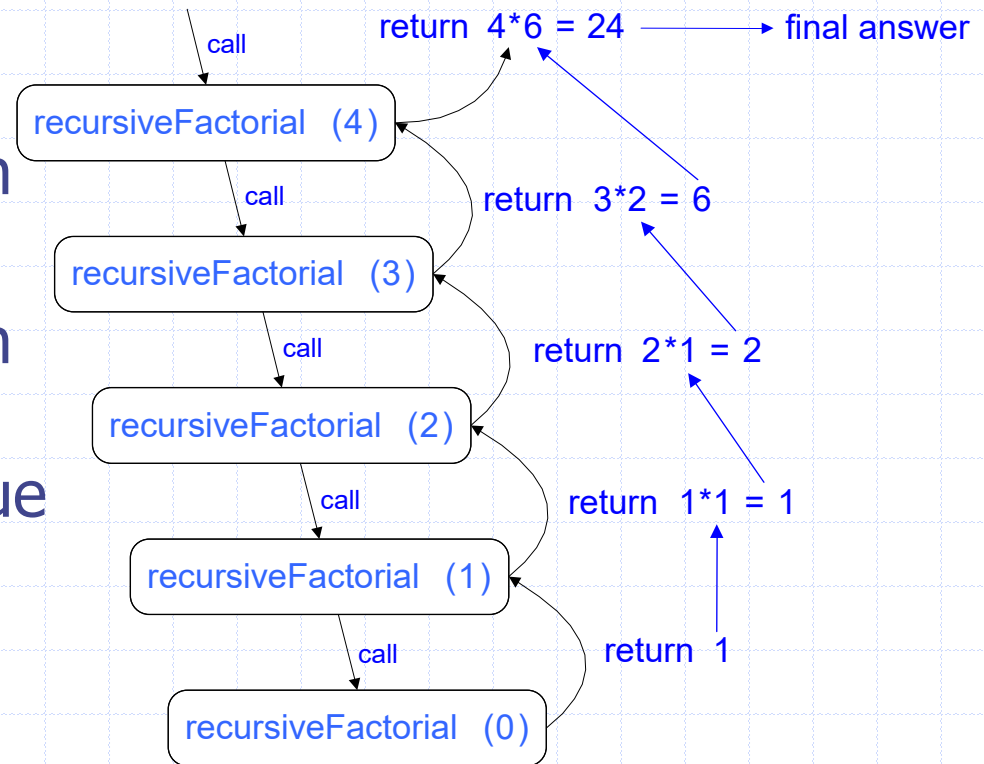
```
1. int factorial(int n) // n >= 0
2. {
3.     if (n == 0)
4.         return 1;
5.     else
6.     {
7.         int smaller = factorial(n-1);
8.         return n * smaller; // or just return n * factorial(n-1)
9.     }
10. }
```

Visualizing Recursion

□ Recursion trace

- A box for each recursive call
- An arrow from each caller to callee
- An arrow from each callee to caller showing return value

□ Example



Linear Recursion

- Test for base cases

- Every possible chain of recursive calls **must** eventually reach a base case.

- Recur once

- Perform a single recursive call
- Might branch to one of several possible recursive calls
- makes progress towards a base case.

Example of Linear Recursion

Algorithm **linearSum**(A, n):

Input:

Array, A, of integers
Integer n such that
 $0 \leq n \leq |A|$

Output:

Sum of the first n
integers in A

if $n = 0$ then

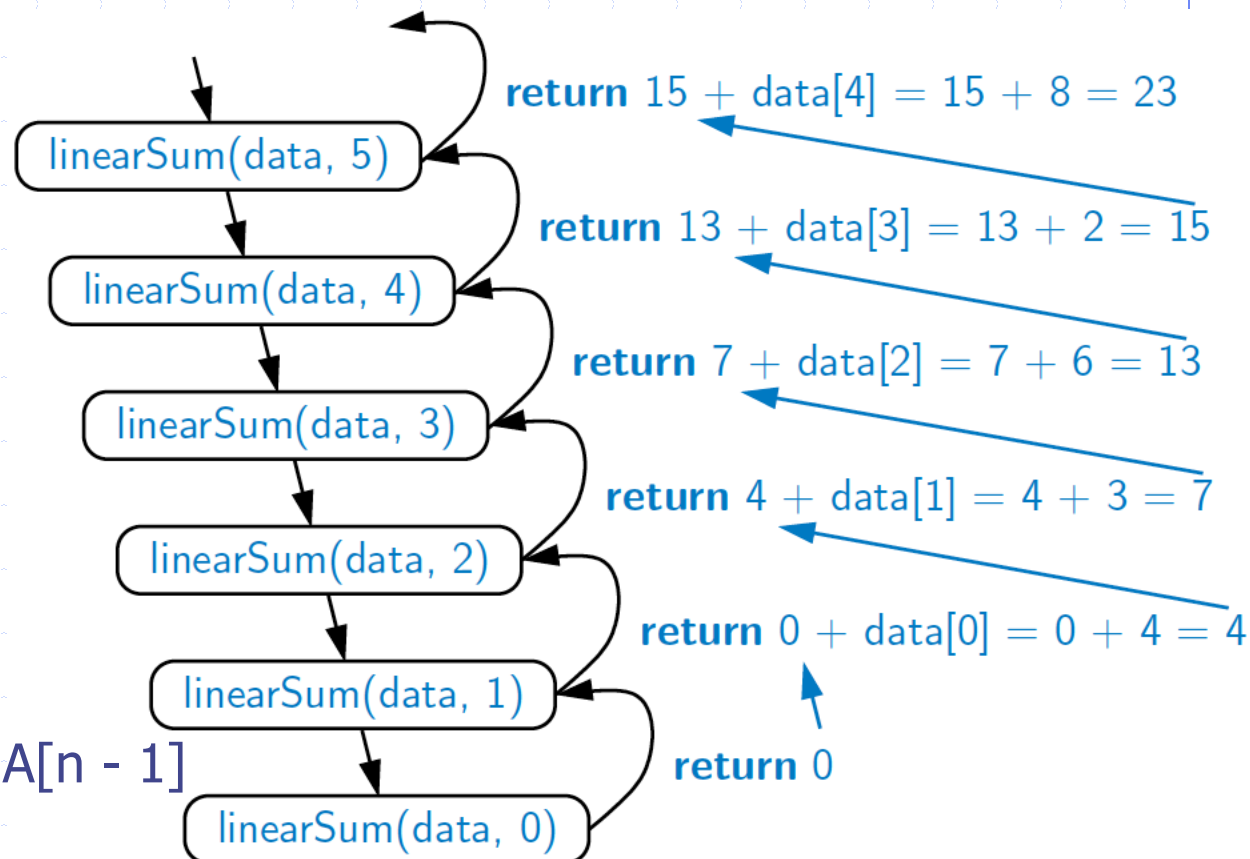
return 0

else

return

linearSum(A, n - 1) + A[n - 1]

Recursion trace of **linearSum**(data, 5)
called on array data = [4, 3, 6, 2, 8]



Example of Linear Recursion

Algorithm **linearSum**(A, n):

Input:

Array, A, of integers
Integer n such that
 $0 \leq n \leq |A|$

Output:

Sum of the first n
integers in A

if $n = 0$ then

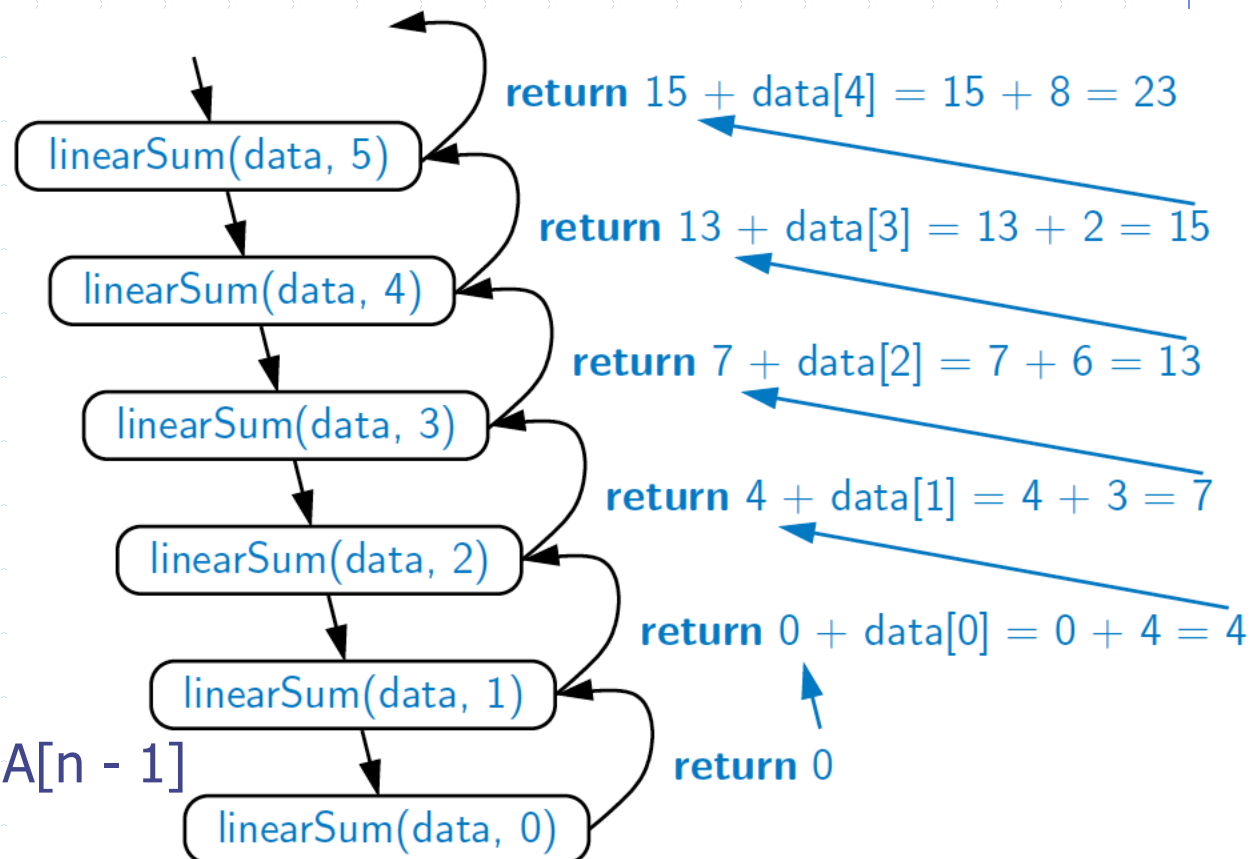
return 0

else

return

linearSum(A, n - 1) + A[n - 1]

Recursion trace of **linearSum**(data, 5)
called on array data = [4, 3, 6, 2, 8]



Reversing an Array

Algorithm `reverseArray(A, low, high)`:

Input: An array `A` and nonnegative integer indices `low` and `high`

Output: The reversal of the elements in `A` starting at index `low` and ending at `high`

Reversing an Array

Algorithm `reverseArray(A, low, high)`:

Input: An array `A` and nonnegative integer indices `low` and `high`

Output: The reversal of the elements in `A` starting at index `low` and ending at `high`

if `low >= high` then return

Swap `A[low]` and `A[high]`

`reverseArray(A, low + 1, high - 1)`

Tail Recursion

- linearly recursive method makes its recursive call as its last step.
- The array reversal method is an example.
- Such methods can be easily converted to non-recursive methods (which saves on some resources).
- Example:

Algorithm *IterativeReverseArray*(A, low, high):

Input: An array A and indices low and high

Output: The reversal of the elements in A starting at index low and ending at high

while low < high **do**

 Swap A[low] and A[high]

 low = low + 1

 high = high - 1

return

Binary Recursion

- **two** recursive calls for each non-base case.

Binary Recursion

- Problem: add all the numbers in an integer array A:

Algorithm BinarySum(A, i, n):

Input: An array A and integers i and n

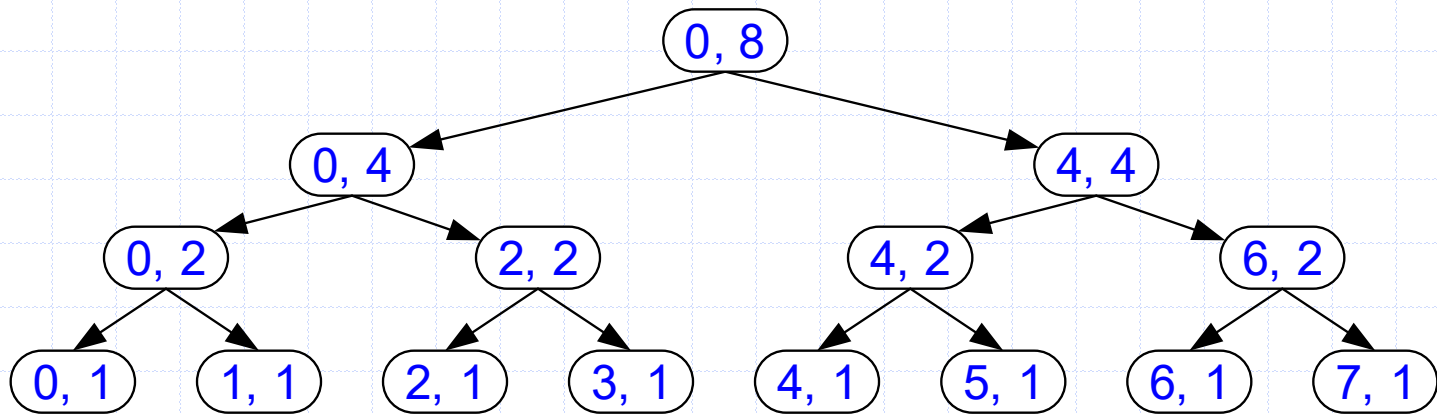
Output: The sum of the n integers in A starting at index i

if n = 1 **then**

return A[i]

return BinarySum(A, i, n/ 2) + BinarySum(A, i + n/ 2, n/ 2)

- Example trace:



Binary Recursion

- Problem: add all the numbers in an integer array A:

Algorithm BinarySum(A, i, n):

Input: An array A and integers i and n

Output: The sum of the n integers in A starting at index i

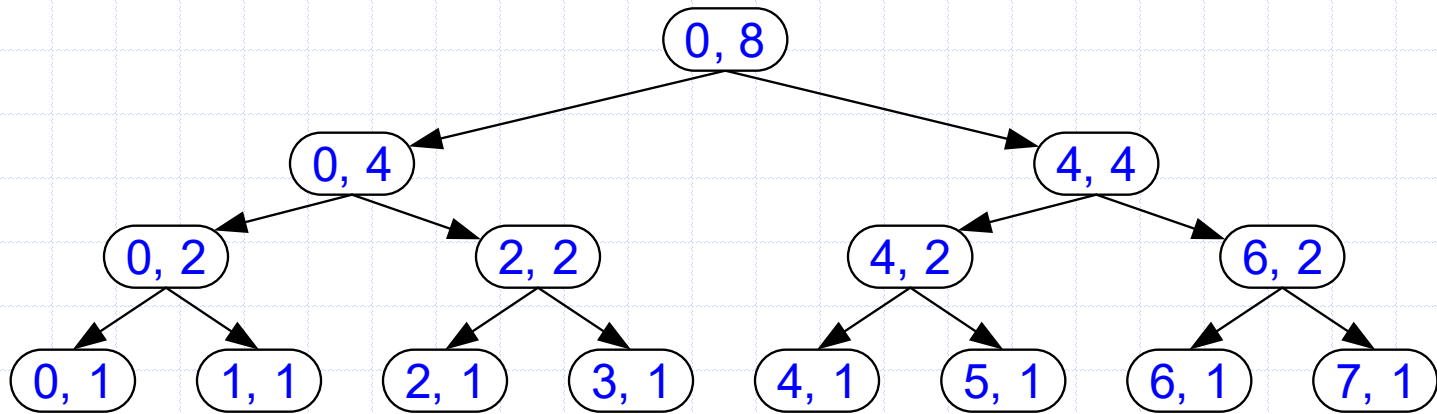
if $n = 1$ **then**

return A[i]

return BinarySum(A, i, $n/2$) + BinarySum(A, i + $n/2$, $n/2$)

Decomposition?
Base case?
Composition?

- Example trace:

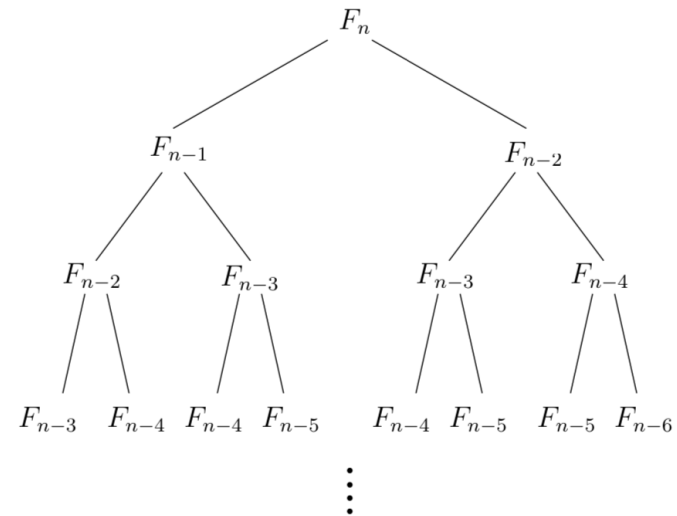


Fibonacci Numbers

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0. \end{cases}$$

```
function fib1(n)
if n = 0: return 0
if n = 1: return 1
return fib1(n-1) + fib1(n-2)
```

- Running time of fib1?



$$T(n) = T(n-1) + T(n-2)$$

How fast does $T(n)$ grow?

Fibonacci Numbers

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0. \end{cases}$$

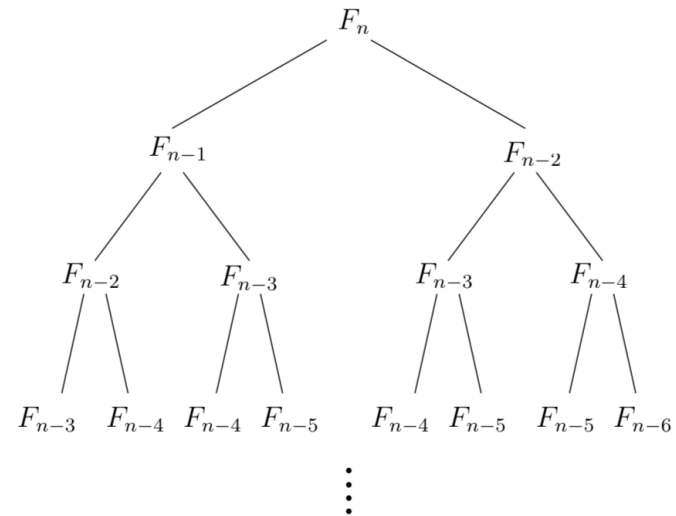
function fib1(n)

if $n = 0$: return 0

if $n = 1$: return 1

return fib1($n-1$) + fib1($n-2$)

- Running time of fib1?



$$T(n) = T(n-1) + T(n-2)$$

$$\left(\frac{3}{2}\right)^n \leq T(n) \leq 2^n$$

A better algorithm

```
function fib2(n)
  if  $n = 0$  return 0
  create an array  $f[0 \dots n]$ 
   $f[0] = 0, f[1] = 1$ 
  for  $i = 2 \dots n$ :
     $f[i] = f[i-1] + f[i-2]$ 
  return  $f[n]$ 
```

– Running time?

– Can you do better?

Algorithm Binary Search (A, n, T)

Inputs: A is an array of n sorted elements.
 T is the element value we want to locate

Output: index of the array element with value T .
if it is present, else return NIL.

BINARY SEARCH

1	2	3	4	5	6	7	8	9	10	11
2	5	8	12	16	23	46	59	60	150	200

					23					
--	--	--	--	--	----	--	--	--	--	--

--	--	--	--	--	--	--	--	--	--	--

↓

BinarySearch(A, v, low, high)

If $\text{low} > \text{high}$ Return -1

$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$

If $A[\text{mid}] == v$

Return mid

If $A[\text{mid}] > v$

Return BinarySearch(A, v, low, mid - 1)

Else

Return BinarySearch(A, v, mid + 1, high)

ITERATIVE-BINARY-SEARCH (A, v)

```
1   $low = A[1]$ 
2   $high = A[A.length]$ 
3  while  $low \leq high$ 
4       $mid = \lfloor (low + high) / 2 \rfloor$ 
5      if  $v == A[mid]$ 
6          return  $mid$ 
7      elseif  $v > A[mid]$ 
8           $low = mid + 1$ 
9      else
10          $high = mid - 1$ 
11 return NIL
```

— What is the time
Complexity of Binary
Search? $O(\log n)$

— Does it change with
the data structure used
to implement A ?

Summary

□ 3 components of recursion

- Decomposition (smaller problems)
- Base case (smallest problem with known solution)
- Composition (solution from smaller solutions)

Examples	Smaller	# of smaller problems
Factorial	-1	1
ArraySum	-1	1
Reverse array	-2	1
Fibonacci	-1, -2	2
BinarySum	1/2	2
BinarySearch	1/2	1